ID and Name:	UC-01: API Gateway Routing & Verification
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	The Gateway acts as the single entry point (Port 5000). It intercepts all requests, validates the API Key, verifies User Tokens (if present), and routes the request to the appropriate microservice.
Trigger:	Any HTTP Request sent to http://localhost:5000
Preconditions:	PRE-1: Gateway Service is running on Port 5000. PRE-2: User Service is running to verify tokens.
Postconditions:	POS-1: Request is successfully forwarded to the downstream service. POS-2: User identity headers (X-User-Email, X-User-Role) are injected.
Normal flow:	1. Client sends a request with Header peko-key. 2. Gateway validates peko-key. 3. Gateway checks for Authorization: Bearer <token> header. 4. Gateway calls User Service (/api/auth/verify) to validate token. 5. User Service returns user profile and role. 6. Gateway injects user info into headers. 7. Gateway forwards request to the target service (Movie/Booking/Payment). 8. Gateway returns the response from the target service to the client.
Alternative Flows:	4a. Invalid Token: If User Service returns 401, Gateway denies request immediately.
Exceptions:	2a. Invalid API Key: Returns 401 Unauthorized.

	5a. User Service Down: Returns 500 Internal Server Error.
Priority:	High

ID and Name:	UC-02: User Registration
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	User
Description:	Allows a new user to create an account in the system.
Trigger:	Guest sends: POST /api/auth/register
Preconditions:	PRE-1: Email must not already exist in users.db.
Postconditions:	POS-1: New user record created with role 'user'.
Normal flow:	1. User submits email and password. 2. Gateway forwards to User Service. 3. User Service hashes the password. 4. User Service saves the record to users.db. 5. System returns success message.
Alternative Flows:	none
Exceptions:	4a. Email Exists: Returns 400 Bad Requests.
Priority:	High

ID and Name:	UC-03: User Login
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	Authenticates a user and issues a Bearer Token (JWT) for accessing protected resources.

Trigger:	User sends POST /api/auth/login
Preconditions:	PRE-1: User/admin account must exist.
Postconditions:	POS-1: System returns a valid Access Token. POS-2: Client stores this token for future requests.
Normal flow:	1. User submits email and password. 2. Gateway forwards request to User Service. 3. User Service verifies email exists. 4. User Service validates password hash. 5. User Service generates a Token containing user ID and Role. 6. System returns the Token to the user.
Alternative Flows:	none
Exceptions:	3a. User Not Found / Wrong Password: Returns 401 Unauthorized.
Priority:	High

ID and Name:	UC-04: Create Movie
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Adds a new movie title to the system database.
Trigger:	Admin sends POST /api/movies
Preconditions:	PRE-1: User has 'admin' role. PRE-2: Movie ID must be unique.
Postconditions:	POS-1: New movie record is inserted into movies.db.

Normal flow:	1. Admin submits movie details (ID, Title, Genre, Duration). 2. Gateway checks Admin Token. 3. Movie Service checks if ID already exists. 4. Movie Service inserts data into movies table. 5. System returns "Movie Created" message.
Alternative Flows:	none
Exceptions:	3a. Duplicate ID: Returns 400 Bad Request.
Priority:	High

ID and Name:	UC-05: Update Movie
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Allows Admin to correct or update movie information (Title, Genre, Duration).
Trigger:	Admin sends PUT /api/movies/{id}
Preconditions:	PRE-1: Movie exists. PRE-2: User has 'admin' role.
Postconditions:	POS-1: Movie details updated in movies.db.
Normal flow:	1. Admin submits new details (e.g., fix typo in Title). 2. Gateway verifies Admin Token. 3. Movie Service updates the record in database. 4. System returns success message.

Alternative Flows:	none
Exceptions:	none
Priority:	Medium

ID and Name:	UC-06: Cascade Delete Movie
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Deletes a movie and automatically deletes all associated showtimes and refunds all booked tickets.
Trigger:	Admin sends DELETE /api/movies/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Movie ID must exist.
Postconditions:	POS-1: Movie, Showtimes, and Bookings are deleted. POS-2: Refund Events are sent.
Normal flow:	1. Admin requests deletion. 2. Movie Service finds all associated showtimes. 3. Loop: Trigger Refund Process for each showtime. 4. Delete all showtimes. 5. Delete the movie record. 6. Returns the count of refunded tickets.
Alternative Flows:	none

Exceptions:	none
Priority:	Medium

ID and Name:	UC-07: Create Showtime
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Schedule a movie and generate seats.
Trigger:	Admin sends POST /api/showtimes
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Movie ID must exist. PRE-3: Time is valid.
Postconditions:	POS-1: Showtime saved in movies.db. POS-2: Seats created in booking.db.
Normal flow:	1. Admin sends time and price. 2. Movie Service saves showtime. 3. Movie Service calls Booking Service (/internal/seats) to generate seats. 4. Returns success.
Alternative Flows:	none
Exceptions:	none
Priority:	High

ID and Name:	UC-08: Update Showtime & Auto-Refund
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Update showtime. If price changes, existing bookings are automatically cancelled and refunded.
Trigger:	Admin sends PUT /api/showtimes/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Showtime ID must exist.
Postconditions:	POS-1: Update successful. POS-2: (If price changed) Tickets cancelled, Refund Event sent.
Normal flow:	1. Admin changes price. 2. Movie Service detects price change. 3. Calls Booking Service to cancel all tickets. 4. Publishes REFUND_NOTIFICATION to RabbitMQ. 5. Updates new price in DB.
Alternative Flows:	none
Exceptions:	none
Priority:	High

ID and Name:	UC-09: Delete Showtime
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Delete a specific showtime and refund sold tickets.
Trigger:	Admin sends DELETE /api/showtimes/{id}

Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Showtime ID must exist.
Postconditions:	POS-1: Showtime deleted. POS-2: Booked tickets are refunded.
Normal flow:	1. Admin requests deletion. 2. Gateway verifies Admin Role. 3. Trigger Refund Process. 4. Delete record in DB. 5. Returns success message.
Alternative Flows:	none
Exceptions:	none
Priority:	High

ID and Name:	UC-10: Search Movies
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	List all movies or search by title or show one movies by id
Trigger:	GET /api/movies (with ?query=...) GET /api/movies/{id}
Preconditions:	PRE-1: Movie Service is running. PRE-2: Database has movie records (optional).

Postconditions:	POS-1: Returns list of movies matching query. POS-2: No database modification.
Normal flow:	1. User sends query. 2. Service performs LIKE search. 3. Returns list.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

ID and Name:	UC-11: View Showtimes Schedule
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	View list of available showtimes and prices.
Trigger:	GET /api/showtimes
Preconditions:	PRE-1: Movie Service is running.
Postconditions:	POS-1: Returns list of showtimes. POS-2: No database modification.
Normal flow:	1. User requests schedule. 2. System returns list of showtimes.
Alternative Flows:	none
Exceptions:	none

Priority:	High

ID and Name:	UC-12: Book Ticket
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	User
Description:	User reserves a seat. Price is fetched dynamically.
Trigger:	POST /api/bookings
Preconditions:	PRE-1: User must be logged in. PRE-2: Seat must be 'available'.
Postconditions:	POS-1: Booking created. POS-2: OrderPlaced event sent.
Normal flow:	1. User selects seat. 2. Service checks availability. 3. Calls Movie Service to fetch price. 4. Creates booking and updates seat status. 5. Sends event to RabbitMQ.
Alternative Flows:	none
Exceptions:	2a. Seat Taken: Returns 400 Bad Request.
Priority:	High

ID and Name:	UC-13: View Bookings
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026

Primary Actor:	User
Description:	View history of bookings made by the user.
Trigger:	GET /api/bookings
Preconditions:	PRE-1: User must be logged in.
Postconditions:	POS-1: Returns list of bookings for that user.
Normal flow:	1. Gateway extracts email from Token. 2. Service filters bookings by email. 3. Returns list.
Alternative Flows:	none
Exceptions:	none
Priority:	High

ID and Name:	UC-14: View Booking Detail
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	User
Description:	View specific details of a single ticket.
Trigger:	GET /api/bookings/{id}
Preconditions:	PRE-1: User must be logged in. PRE-2: User must own the ticket.
Postconditions:	POS-1: Returns ticket details.
Normal flow:	1. User requests ID. 2. System verifies ownership. 3. Returns details.

Alternative Flows:	none
Exceptions:	2a. Not Owner: Returns 403 Forbidden.
Priority:	High

ID and Name:	UC-15: Cancel Booking (Manual)
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	User
Description:	User cancels their own ticket.
Trigger:	DELETE /api/bookings/{id}
Preconditions:	PRE-1: User must be logged in. PRE-2: User must own the ticket.
Postconditions:	POS-1: Booking deleted. POS-2: Seat becomes available.
Normal flow:	1. User requests cancellation. 2. System verifies ownership. 3. Deletes booking. 4. Frees up seat.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

ID and Name:	UC-16: Process Payment
Create by:	Nguyễn Gia Bảo / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Pay for a ticket and generate an invoice.
Trigger:	POST /api/payments
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: Valid Booking ID exists.
Postconditions:	POS-1: Payment logged. POS-2: InvoiceGenerated event sent.
Normal flow:	1. User submits Booking ID. 2. Gateway verifies Token and routes to Payment Service. 3. Payment Service calls Booking Service (Internal API) to verify the booking exists and get the correct amount. 4. Payment Service processes the mock transaction. 5. Payment Service publishes InvoiceGenerated event to RabbitMQ. 6. System returns "Payment Successful" message to User.
Alternative Flows:	none
Exceptions:	3a. Booking Not Found: Payment Service returns 404. 3b. Already Paid: Payment Service returns 400 Bad Request.
Priority:	High